Appendix:

Recursive Self-Improvement for Robotic Manipulation in RoboDojo

AgiBot Research Team

1 Introduction

Success in robotic manipulation depends on decisions made across multiple stages. Target localization errors propagate to grasping and placement; previously placed objects change the space available for later motions; and recovery consumes the remaining control budget. Local events such as gripper closure or arm arrival do not establish task completion. Progress must still be checked through readings after lifting, images after placement, and subsequent execution. Improving task performance therefore requires tracing how a failure develops through the execution sequence, rather than only correcting its final manifestation.

Executable programs provide an inspectable and editable representation of these decisions. Code as Policies and related robot-agent systems compose perception and control capabilities through programs [1, 2]. This representation makes observation timing, target estimation, operation order, retry conditions, and budget allocation explicit, while separating task-specific logic from shared tools. A revision can thus be tied to a particular decision, and its effects can be checked across other stages or tasks that use the same tool.

Editability alone, however, does not ensure improvement. A single failed episode provides a clue; an explanation must account for a group of similar failures and distinguish observed behavior from inferred causes. Differences in scene arrangement, incidental contacts, and interrupted execution can make two runs difficult to compare, while improved local checks may still accompany failure of the complete task. The development process must therefore connect failure diagnosis, version comparison under matched conditions, and the retention of experience. It checks whether the intended behavior occurs and previously successful episodes remain successful, while recording unsupported explanations to avoid repeating the same attempts.

We introduce a novel recursive self-improvement (RSI) workflow in which a coding agent autonomously constructs, diagnoses, revises, and validates task programs. The agent locates the earliest observable deviation in execution records, groups failures by mechanism, prioritizes a problem, and develops a candidate version from a testable causal hypothesis. The candidate and its parent are rerun with matched scene arrangements, configurations, and budgets. Complete-task success rates, scores, and newly observed failures determine whether the revision is effective. Subsequent iterations inherit both the retained solution and its development evidence, allowing execution to change both future behavior and the basis for further diagnosis.

The workflow separates execution within an episode from development between episodes. The running program uses robot observations to maintain state, check progress, and select recovery actions; after the episode, the coding agent analyzes execution records and evaluation results. No simulator ground truth is used at any stage: object recognition and manipulation decisions are based on robot observations. Humans specify task objectives, information boundaries, and budgets, and authorize final integration. Content-based version identifiers associate each episode's

results with the code that actually ran. Formal evaluation uses a frozen configuration and verifies scene coverage and version consistency, making each score traceable to its execution solution.

The resulting system achieves a mean success rate of 56.42% across five dimensions and an overall score of 61.90 on the RoboDojo simulation benchmark, exceeding PyRUA-Lean's 50.35% and 55.26, respectively. These results represent the best overall performance among the compared methods (SOTA; Table 2).

2 Task Setting and Program Execution

2.1 Task Requirements and Observation Interface

RoboDojo evaluates tabletop manipulation across five dimensions: memory, precision, open semantics, generalization, and long-horizon execution [3]. We use an ARX-X5 dual-arm robot. Each task is defined by an instruction, completion criteria, permitted information and tools, and a control-step budget. Development begins by specifying the required object relationships, dependencies between operations, and finalization requirements, which determine the program's stages and checks. Tolerances, action budgets, and completion criteria are recorded separately for each task.

At runtime, the program reads RGB images from the head and two wrist cameras, joint angles, gripper readings, end-effector link (`link6`) poses, and the task instruction from the execution context. It writes joint and gripper targets for the next control step. Perception and motion tools use these inputs to estimate targets and control manipulation. Table 1 summarizes the interfaces for observation, control, and execution records.

Table 1: Observation, control, and recording interfaces for task programs. An external evaluator produces benchmark results for analysis and version comparison after execution.

Component	Interface or artifact	Role
Visual observations	`ctx.rgb(cam)`	RGB images from head and wrist cameras.
Robot state	`ctx.q(arm)`, `ctx.g(arm)`, `ctx.ee(arm)`	Joint and gripper states, and end-effector link poses.
Task instruction	`ctx.instruction`	Goal and task-specific requirements.
Control targets	`ctx.cmd_q[arm]`, `ctx.cmd_g[arm]`	Joint and gripper targets for the next control step.
Control tools	`move`, `goto`, `grip`, `hold`, `move_joints`	Motion sequences that advance control through `yield`.
Execution records	`ctx.log`, `ctx.snap`, `ctx.failure`	Stage events, evidence for decisions, key images, and termination reasons.
External evaluation	Episode success and score from the evaluator	Complete-task outcomes for diagnosis and comparison.

No simulator ground truth is used during initial program construction, execution, failure diagnosis, or candidate validation. This includes simulator-provided object dimensions, geometric models, contours, poses, and internal scene states. Object recognition and position estimation rely on robot observations; simulation assets are not accessed, and simulator ground truth is not used to validate perception results.

Episode success and scores from the benchmark evaluator support analysis and version comparison after execution. They are not inputs to target localization or action selection during execution. Layout identifiers and evaluation seeds are used only to organize trials and align comparisons,

and do not influence task-program decisions. Evaluation results are stored separately from the program's local judgments based on current observations.

2.2 Program Representation and Stepwise Execution

A task solution comprises a `run(ctx)` entry point, task-specific helper functions, shared tools, required data, and execution configuration. The task layer specifies target selection, operation order, retries, and recovery; the shared layer provides coordinate transformations, perception processing, kinematics, and motion primitives. Task-specific thresholds, offsets, and target-position rules remain in the task layer so that modifying one task does not affect other users of shared tools.

`run(ctx)` is a Python generator. Computational helpers return estimates or plans without advancing simulation. Motion helpers update joint and gripper targets and relinquish control through `yield`. The execution environment sends the command, updates the context with the next observation, and resumes the program from its suspension point. Motion routines may also return local outcomes that determine subsequent branches. This allows the program to observe and reassess between actions without specifying the entire episode's action sequence in advance.

Local variables and `ctx.memory` retain the current stage, observed objects, completed operations, and retry counts. For example, before covering blocks, the program records the correspondence between colors and positions and later uses it to determine the uncovering order despite occlusion. Each environment reset creates a new episode context, while task programs and development records persist across iterations. Figure 1 illustrates the program structure from observation and action to progress checks and finalization. Control handoff to an action policy and its iterative refinement are described in Section 4.

```
def run(ctx):
    yield from ready(ctx)
    state = initialize_from_observations(ctx)
    while has_unfinished_work(state):
        if not enough_steps_for_attempt(ctx, state):
            ctx.failure = "insufficient remaining steps"
            break
        target = estimate_target(ctx, state)
        if not target.valid:
            ctx.failure = "target not reliably localized"
            break
        plan = choose_feasible_operation(ctx, target, state)
        if plan is None:
            ctx.failure = "no feasible operation"
            break
        yield from execute_operation(ctx, plan)
        outcome = check_progress(ctx, state)
        ctx.log("stage", outcome=outcome.label)
        if outcome.confirmed:
            update_progress(state, outcome)
        elif recovery_available(ctx, state):
            yield from recover(ctx, state)
        else:
            ctx.failure = "local recovery exhausted"
            break
    if has_unfinished_work(state) and handoff_available(ctx):
        yield request_policy_handoff(ctx)
        return
    yield from finalize(ctx, state)
```

Figure 1: Schematic task-program structure. Helper names are illustrative. Motion routines update control targets and advance through `yield`; checks use the resulting observations. Planning-exception handling within motion and recovery routines is omitted: the task program must catch these exceptions and enter recovery or termination branches. When an action policy is enabled, control is explicitly handed over before finalization. Otherwise, the program releases objects, returns the arms, and waits for stabilization as required by the task.

3 Recursive Improvement Workflow

Figure 2 summarizes the cycle. The program specifies how the next episode will be executed, while development records inform the next revision; both are retained across rounds. Data partitions, permitted information, and evaluation coverage are reported according to the records of each experiment.

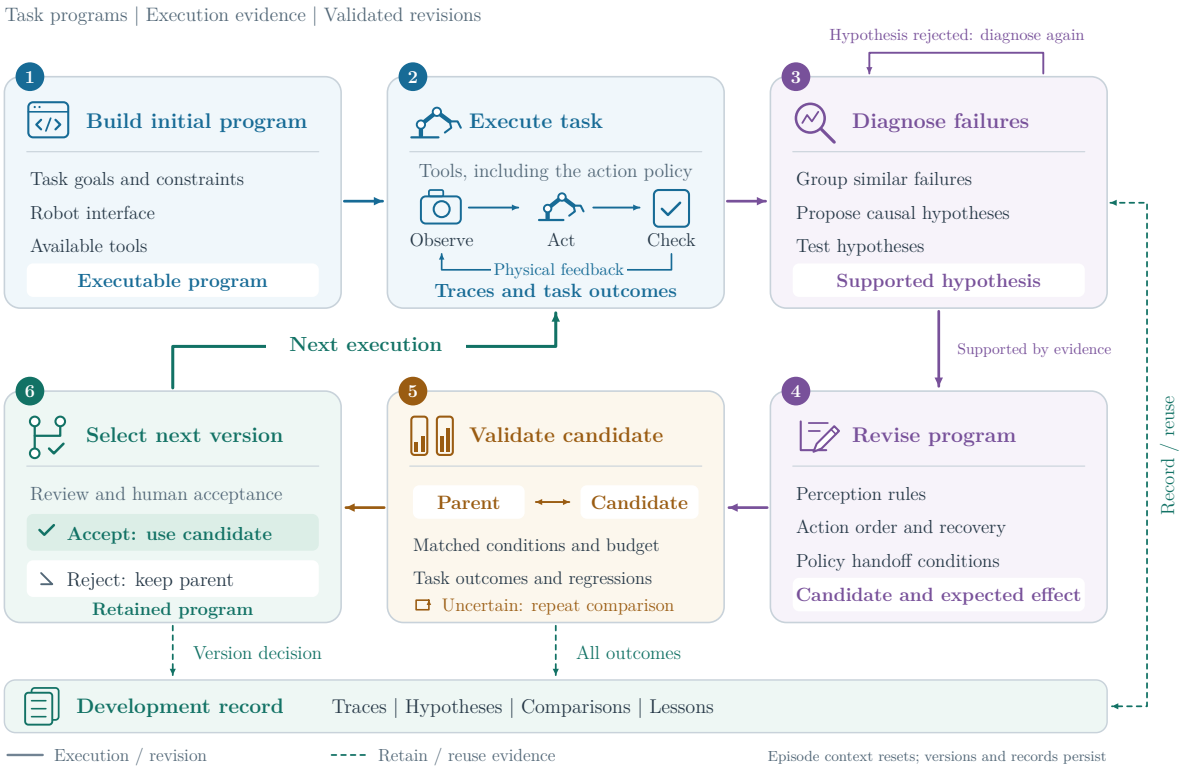


Figure 2: Recursive improvement of RoboDojo task programs. The coding agent constructs a solution, executes the task, diagnoses failures, and tests candidate revisions. Parent and candidate versions are compared under matched task conditions; inconclusive results require additional evidence. After review and final acceptance, the selected version is used for the next episode. Development records persist for later diagnosis, including evidence from revisions that were not accepted.

3.1 Initial Program Construction and Baseline Execution

The initial program translates the task objective into stages that can be executed and checked. The coding agent first identifies the required object relationships, dependencies between operations, and observable conditions for completing each stage. It then selects perception and motion tools to implement these stages.

Initial perception can be developed through an observation-gathering routine. The routine moves both arms out of the main camera view and saves head and wrist images across a set of development layouts. The agent checks whether extracted target regions agree with the images, then uses actual execution to determine whether the estimates support grasping, transport, and placement. Operation sequences and recovery branches must satisfy their preconditions and reserve control steps for subsequent checks and finalization. Initial program development thus addresses observation availability, action feasibility, and complete-task execution in sequence.

Early runs establish the initial version's baseline. The coding agent first checks that the program loads, continues to advance its actions, and completes finalization, while recording both complete-

task scores and success rates. For an existing solution, development resumes from the retained version and its records. Results are used for subsequent attribution and comparison only after episodes have completed and their outcomes have been associated with the versions that produced them. Interrupted evaluations or missing results must be resolved before they are treated as evidence of program failure.

3.2 Execution Evidence and Episode-Level Version Tracking

Evidence collection connects the program's decisions to task outcomes. The coding agent runs the program through the evaluation entry point under specified task configurations and layouts, recording stage events, target estimates, gripper readings, tracking errors, retry decisions, and termination reasons. Images are saved at key stages. The evaluator separately reports episode success and task-progress scores. Execution records explain why the program selected an operation; complete-task results determine whether those operations ultimately achieved the objective.

Version records contain the code commit, a content identifier, and the execution configuration. The content identifier is a hash of the code files actually executed, distinguishing versions whose names are identical but whose contents differ. Both the task program and its shared dependencies are recorded. A run manifest associates this identifier with the task, actual layout, and episode, linking scores, termination information, stepwise logs, and images. When an action policy is enabled, its model and inference configuration are also recorded. Existing results can therefore be traced to the execution solution that produced them even after subsequent code changes.

Records must also distinguish early program termination, exhaustion of the execution budget, and evaluator errors. The coding agent uses the program's final step, failure reason, and evaluation status to identify where execution stopped. An environment reaching its final step limit does not, by itself, establish that the program remained active until that point. Likewise, completion of a local operation only indicates that the program passed its own checks; it does not replace the evaluator's complete-task success judgment.

3.3 Failure Prioritization and Hypothesis Testing

Diagnosis begins by selecting the failure pattern to address. The coding agent groups failures by stage and observed behavior, counts their occurrences, and selects representative episodes. Priority is based on the product of occurrence count and score loss per episode. Iteration addresses programs that cannot run, systematic recognition errors, and recurring manipulation failures before isolated issues or small score differences. This concentrates the evaluation budget on mechanisms that can explain a group of failures.

For the selected pattern, the coding agent examines termination records, stepwise logs, and key images to locate the earliest observable deviation. Slippage during transport calls for examining gripper readings after the test lift and the subsequent transport sequence. Path deviations require checking consecutive motion solutions and joint tracking. Damage to an existing structure after placement calls for examining descent speed, stability, and release position. Observations and inferred causes are recorded separately, and the agent checks whether the same explanation accounts for the other failures in the group.

Perception diagnosis checks whether the target regions extracted by the program agree with the recorded images. Manipulation diagnosis examines the relationship between action commands, changes in robot state, and subsequent task progress. Competing explanations are investigated through additional observations or controlled execution comparisons. Explanations with insufficient evidence remain hypotheses rather than confirmed causes.

A hypothesis must also identify a modifiable routine and an observable expected change. For example, if an incorrect target region prevents subsequent operations, revising the recognition rule should first recover a valid target and then allow the manipulation stage to begin. When several causes could explain the observations, the coding agent constructs variants from the same parent version that change only the mechanism under investigation, then compares intermediate behavior and complete-task outcomes on the same layouts. Rejected hypotheses return to diagnosis; unresolved hypotheses receive additional runs or remain pending. Both are recorded so that later rounds do not repeat tests of explanations already ruled out.

3.4 Local Revisions and Candidate Construction

A candidate revision translates a mechanism hypothesis into concrete code changes. In an isolated version, the coding agent adjusts target estimation, observation timing, grasp candidates, motion paths, operation order, or recovery branches while leaving unrelated components unchanged. If a shared tool must change, its callers are listed explicitly and affected tasks are included in subsequent comparisons. The candidate record preserves its parent version, supporting episodes, modified locations, expected effects, and unresolved issues.

Revisions can change both actions and the conditions used to assess them. Uncertain perception may call for an additional close-range wrist observation; an unstable grasp may call for a test lift and recheck; an infeasible motion plan may require a different candidate pose or path. Changes to thresholds, offsets, retry limits, and other parameters are documented with their rationale, scope, and validation results. Observation-derived estimates are recorded separately from manually specified execution parameters, and subsequent execution tests whether a parameter change helps complete the task.

Before execution comparisons, the coding agent checks imports, task entry points, resource references, and tests relevant to the change. An independent review checks whether the implementation follows the intended design. Existing images and execution records are used to assess whether the revision addresses the observed problem, after which representative failed episodes are rerun to test the expected change. Passing code checks or producing the intended local behavior does not remove the need for complete-task evaluation.

3.5 Paired Comparisons and Version Retention

The parent and candidate are rerun with the same task configuration, execution budget, and actual layouts. A per-layout table records both versions' scores and success status, identifying newly successful episodes, unresolved failures, and episodes that succeeded before the revision but fail afterward. If the evaluator discards abnormal episodes and substitutes additional scenes, comparisons are realigned using the identifiers of the layouts actually executed, and unmatched results are reported. Identical seeds or requested episode counts alone do not establish that the scenes are the same.

Version retention considers both the intended revision and complete-task performance. The coding agent first checks whether the expected intermediate change occurred, then applies predefined comparison rules to assess whether the evidence supports an improvement in score or success rate. A change with unchanged performance and no new failure patterns may be kept as a candidate, but this does not establish improved task capability. Changes that introduce new failures, contradict the hypothesis, or lack supporting evidence do not replace the parent; their code and comparison records remain in the development history.

Isolated failures and variation in contact dynamics may require broader layout coverage or repeated execution. If a comparison cannot distinguish the effect of a revision from execution variability, the decision remains pending. Additional evidence is collected when the budget per-

mits; otherwise, the existing version remains in use. Each round records what changed, what the evidence supports, and what remains unresolved. A version retained after review and human acceptance becomes the starting point for the next round.

3.6 Frozen Evaluation and Experience Retention

Freezing fixes the selected development solution as an artifact that can be reevaluated independently. Following the predefined development coverage and paired-comparison criteria, task programs, shared dependencies, and required configurations are copied into a separate directory, together with their commit and content identifiers. Formal evaluation uses only this fixed directory and verifies the version associated with each episode. After batched runs, aggregation checks the required layout coverage for duplicates and missing episodes, preventing different code versions or incomplete episode sets from being combined into a single result.

Development runs support diagnosis, parameter revision, and candidate validation; evaluation after freezing assesses the selected solution. Subsequent improvement starts from the retained version, uses new execution records to construct candidates, and freezes a new version for reevaluation. Earlier results remain available, together with a record of which evaluation feedback informed later development. Each batch reports its development and evaluation layouts and their overlap. Freezing a version does not turn layouts already used in development into an independent held-out set.

The next round inherits both an executable solution and development experience. The program retains validated perception, planning, checking, and recovery procedures. The records retain failure groups, measurement evidence, supported and rejected hypotheses, per-layout comparisons, and unresolved questions. Each new episode rebuilds its task context, while versions and external records persist across rounds. Recursive improvement therefore changes both the available execution solution and the questions that subsequent development must observe and test.

4 Control Handoff between Task Programs and Action Policies

The iteration process in Section 3 also refines the division of work between a task program and an action policy. The program uses observations to organize operations, check progress, and attempt local recovery. When it cannot continue its current plan, it can transfer the remaining operations to the action policy. Handoff conditions are part of the task program, so the operations attempted, the time spent on recovery, and the physical state at transfer can all change across versions.

4.1 Handoff within an Episode

Each episode starts with the task program. An ordinary `yield` submits a program-generated control command and resumes execution after a new observation arrives; only an explicit handoff request changes the action source. The program can request handoff when a target cannot be reliably localized, candidate actions or local recovery have been exhausted, or the remaining budget is insufficient to continue its plan. A task without an available code plan can also hand over at the start of the episode. Each request records its reason, trigger location, and a specific explanation. The location must be registered in the task's checkpoint list, making the paths that can transfer control inspectable.

The checkpoint precedes cleanup operations such as opening the gripper or returning the arms after abandoning the plan. The runtime validates the request, pauses the program generator, and discards any uncommitted program action from that request. The action policy then receives the

current camera images, robot state, and task instruction, and continues from the physical state at handoff. Neither the environment nor the remaining step budget is reset. Consequently, previously completed operations, object locations, and objects still held by the grippers affect subsequent execution. This implementation forwards observations and the instruction; internal program-stage variables and `ctx.memory` are not automatically exported as policy inputs.

The policy's action chunks are checked for required fields, dimensions, numerical validity, and control limits before being executed step by step. Handoff is one-way within an episode: the action policy remains in control until the episode ends, and the task program is reloaded after reset. If a policy request fails or returns an invalid action chunk, the runtime generates a hold command from the currently observed joint and gripper state, without automatically resuming the program. Handoff indicates that the current program cannot continue its plan; task success is still determined by the external evaluator.

4.2 Recursive Refinement of Handoff Timing

Handoff timing is determined jointly by program conditions and execution order. The agent examines observations, operations, and recovery attempts before transfer, then traces subsequent task progress to distinguish problems in the preceding program actions, the abandonment condition, or the policy's continuation. Repeating an unsuccessful action, for example, consumes the budget available to subsequent execution, whereas an early handoff may skip operations that the program could still complete. Diagnosis links these observations to the actual trigger path before proposing a revision.

A candidate can revise the number of local retries, the budget required for another attempt, the conditions that trigger handoff, and the observations or operations performed before transfer. If an available recovery branch has not yet run, a candidate can move the checkpoint after that branch is exhausted. If repeated attempts make no progress, it can reduce retries or check the budget earlier to leave more steps for the policy. Changing the preceding operation order also changes the physical state at transfer. Handoff timing therefore cannot be reduced to a fixed control-step index.

The cover-blocks task illustrates how these decisions are expressed in code. Before moving a cup, the program checks the available budget; after a test lift, it uses the gripper reading to check whether the cup is held and permits a bounded number of retries. An unfinished stage then leads to a registered handoff branch. When the final cup has already been released, operation bookkeeping distinguishes the final manipulation from the subsequent retreat, preventing a retreat-planning error alone from triggering handoff. The agent can revise budget conditions, retry limits, and stage checks separately to examine their effects on the transfer point and subsequent execution. This bookkeeping only records that an operation occurred; it does not establish benchmark success.

Retained development records also contain revisions that replace immediate entry-point handoff in some tasks with task-specific operations followed by handoff when local attempts are exhausted. Such revisions change both the work performed by the program and the state encountered by the action policy. They demonstrate that the handoff boundary is editable together with the task program; whether a revision improves performance must be established through complete-episode comparisons.

4.3 Validation and Retention of Handoff Revisions

Validation of handoff candidates follows the process in Section 3, with additional checks on control transfer. Comparisons fix the action policy and its inference configuration, run the parent and candidate under matched task conditions and total execution budgets, and associate handoff

reasons, stages, elapsed steps, boundary observations, and subsequent actions with complete-task success and scores. The agent first checks whether retries, the transfer point, or the handoff state changed as intended, then examines task performance and newly introduced failures.

Different versions may hand over on different episodes, so success rates within their respective handoff subsets cannot establish the effect of a revision. When needed, a comparison can change a single checkpoint or suppress handoff while retaining the other program logic to test a specific hypothesis. Interface checks establish that control changes correctly and returned actions are valid; performance assessment relies on complete-task outcomes under matched conditions. Supported program and handoff revisions enter the next version together, while records of unsupported candidates remain available for subsequent diagnosis.

5 Evaluation Results

The overall evaluation uses a fixed, complete execution configuration and does not combine results from different versions. Development and scored evaluations overlap in their test scenes; these results therefore do not constitute an independent evaluation of held-out generalization.

5.1 Overall RoboDojo Results

RoboDojo groups 42 tasks into memory (6), precision (8), open semantics (8), generalization (12), and long-horizon execution (8). Generalization includes standard and randomized scenarios. We report the benchmark's success rate (SR) and task-progress score. Because the dimensions contain different numbers of tasks, overall SR weights the five dimensions equally rather than averaging over all tasks.

Table 2 compares our complete execution configuration with 20 methods from the benchmark leaderboard. Our system achieves a mean SR of 56.42% and a score of 61.90, compared with 50.35% and 55.26 for PyRUA-Lean. Its SR is 76.00% in precision and 53.50% in long-horizon execution. These results evaluate complete systems with their respective implementations; they do not isolate the contribution of RSI or establish the effect of any individual revision.

Table 2: RoboDojo simulation results. All columns except Score report SR (%); Score measures task progress. Parentheses indicate task counts. Overall SR equally weights the five dimensions; Generalization includes standard and randomized scenarios. The twenty comparison methods are drawn from the official leaderboard, while our results come from the authors' evaluation summary. Method citations identify the corresponding papers or official resources. Bold marks the best value among the listed methods.

Method	Memory	Precision	Open Semantics	Generalization	Long-Horizon	Overall	
	(6)	(8)	(8)	(12)	(8)	SR	Score
Agent-based systems							
PyRUA-Lean [2]	68.33	38.25	**58.25**	**43.17**	43.75	50.35	55.26
HexaShell-Preview [4]	**81.00**	32.00	55.25	34.50	31.50	46.85	53.18
WorldScape-R^{2}SI [5]	63.67	15.50	47.50	35.83	34.25	39.35	46.58
EmbodiedEvo [6]	76.67	16.25	49.25	30.83	21.75	38.95	45.08
PhysicalRSI [7]	46.56	32.50	24.92	15.62	37.33	31.38	36.27
GPT-6-Astra [8]	38.67	4.00	31.00	30.50	8.25	22.48	28.97
World Action Models							
VPP2 [9]	55.56	35.17	4.08	29.50	37.00	32.26	39.26
Awomo-0.5 [10]	41.22	33.50	22.92	23.84	26.75	29.64	35.34
VPP2-Preview [11]	50.33	25.58	5.42	24.62	22.17	25.62	31.40
InternW0-Δ [12]	34.00	23.25	10.17	22.78	29.33	23.91	30.77
Liber-0 Lite [13]	35.44	31.08	2.58	19.11	32.92	24.23	29.24
MachEmbodied-U0 [14]	6.67	16.67	3.17	8.78	22.25	11.51	17.78
OpenWAM-α [15]	9.11	9.25	1.08	14.83	25.33	11.92	17.18
Hybrid policy systems							
Liber-0 Preview [16]	37.33	33.17	5.33	18.61	33.17	25.52	30.74
ME-Brain-1.0 [17]	22.78	15.92	5.33	15.34	20.58	15.99	21.67
VLA models							
Simate-beta [18]	33.00	26.92	8.50	27.95	43.42	27.96	33.95
Rex-M1 Preview [19]	48.22	23.83	6.08	26.05	33.00	27.44	33.79
DM0.5 [20]	47.44	16.75	2.08	10.95	19.50	19.34	24.90
GalaxeaVLA (G0.5) [21]	7.33	20.42	1.58	12.83	32.25	14.88	20.23
Xiaomi-Robotics-1 [22]	6.56	18.83	3.58	17.00	23.67	13.93	20.07
Ours	63.67	**76.00**	55.25	33.67	**53.50**	**56.42**	**61.90**

5.2 Program Changes through Recursive Self-Improvement

During development, the agent revises task programs using execution records and tests the changes by running them again. Figure 3 shows score trajectories for nine tasks, each developed over 6–32 rounds. Building a tower and pressing buttons by number reach a score of 100 in multiple later rounds, while stacking bowls improves substantially after a series of low scores. General pickup continues to exhibit many unresolved failures. The declines in these curves show that revisions do not necessarily improve performance: their effects must be assessed through program changes, observed behavior, and complete-task outcomes.

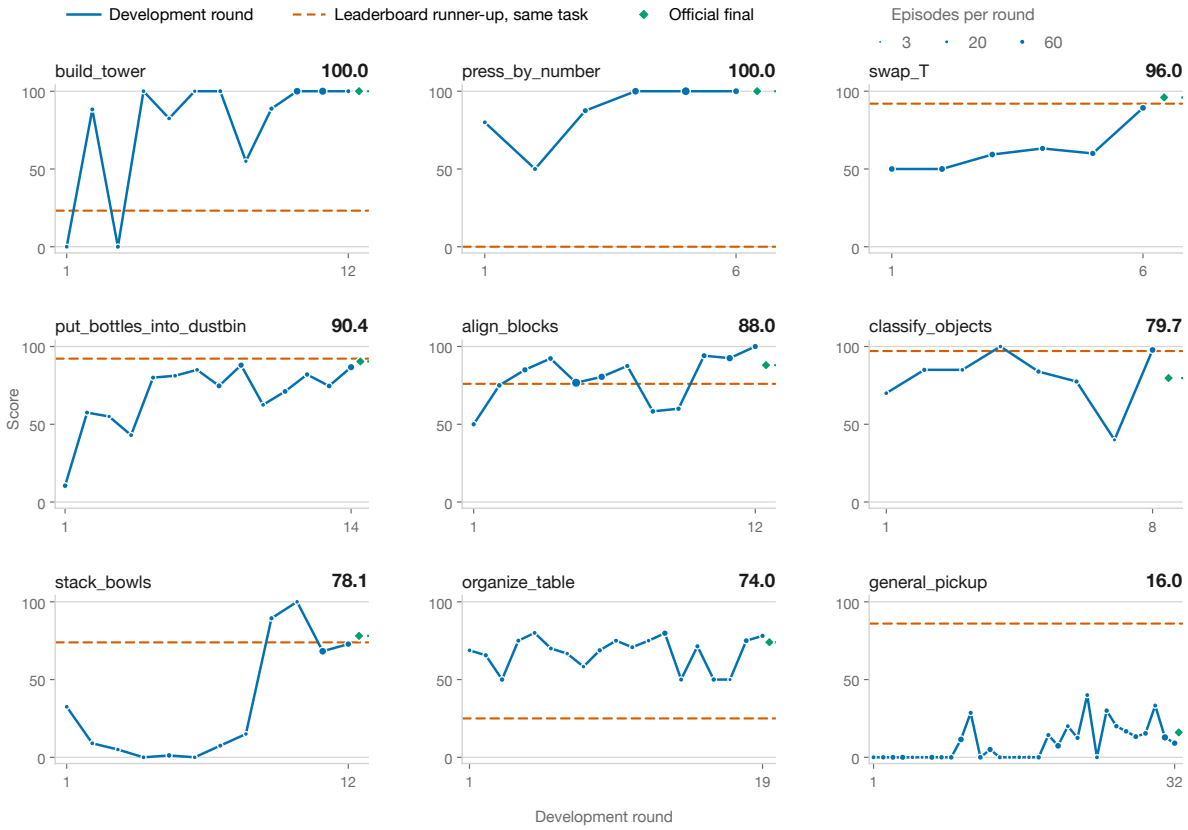


Figure 3: RSI development curves for nine tasks. The vertical axis reports task-progress score and the horizontal axis development round. Blue lines connect recorded rounds, with marker size indicating the number of evaluated episodes. Orange dashed lines show the leaderboard runner-up's score on the same task. Green diamonds and the values in the upper-right corners denote official final evaluation results. Evaluation counts vary across rounds, so the curves describe development trajectories. Score measures task progress and is distinct from complete-task success rate (SR).

Development videos show how these revisions change execution. In the block-alignment task, an earlier version of the program already attempted to straighten the blocks before pushing them into line. However, the middle block remained misaligned after the straightening action, and subsequent pushing with the ruler did not correct its orientation. The agent revised the grasp to grip two side faces of the block and rotate it in place before pushing with the ruler. Figure 4 shows that the middle block remained misaligned with the ruler at the end of the original execution. After the revision, all three blocks were correctly oriented and aligned, and the score in this comparison increased from 0 to 100. The revision preserved the operation sequence while changing the gripper's contact with the block during straightening.

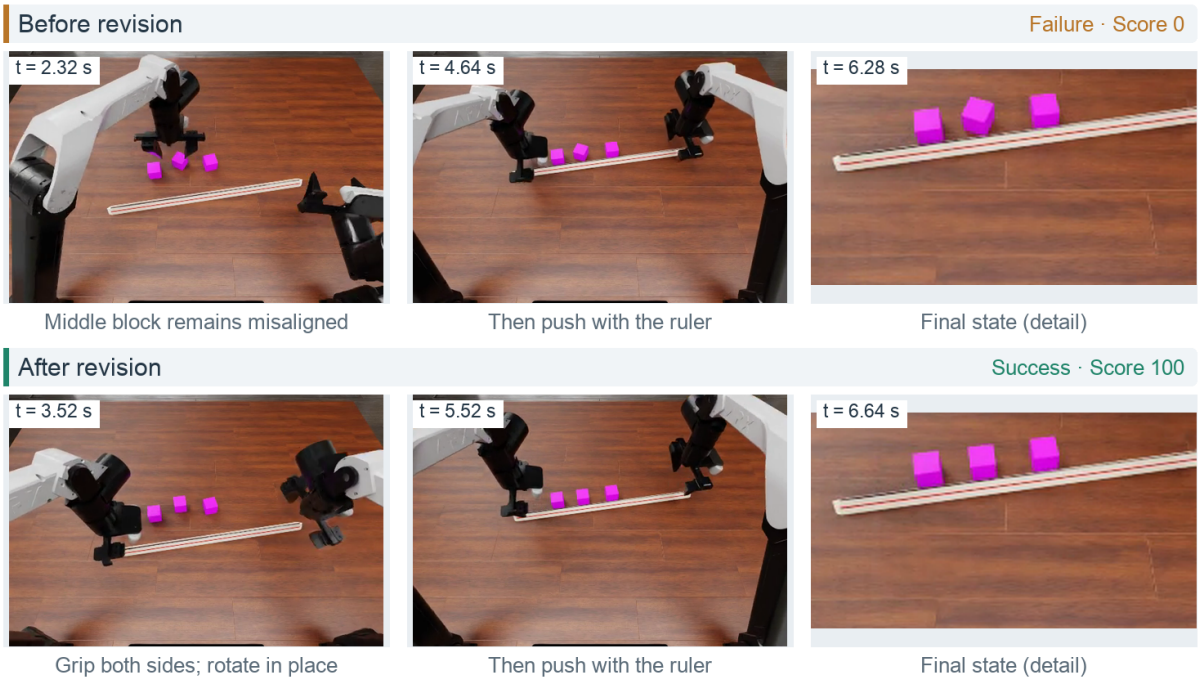


Figure 4: Execution before and after the grasp revision. The top and bottom rows show the original and revised programs, respectively. The left column shows the state after straightening, the middle column shows pushing with the ruler, and the right column enlarges the final block arrangement. Frames are cropped from the original development videos; timestamps refer to those videos, and scores come from the corresponding evaluation records.

Revisions can also change action timing. In the block-swapping task, the original program moved the blocks to their target regions but did not complete the button operation and return both arms to their required positions within the 700-step budget. The agent preserved the swapping procedure, test lifts, visual checks, and path checks, while reducing the final support wait from 10 control steps to 6. Joint readings continued to be checked for motion lag, and the shorter wait reserved steps for the remaining actions. In Figure 5, the original program reached 700 steps without success, whereas the revised program completed the task in 697 steps. Across three comparison trials, the original and revised programs succeeded twice and three times, respectively. Their final video frames look similar; the difference in completion must therefore be interpreted together with the waiting intervals and task-termination records.

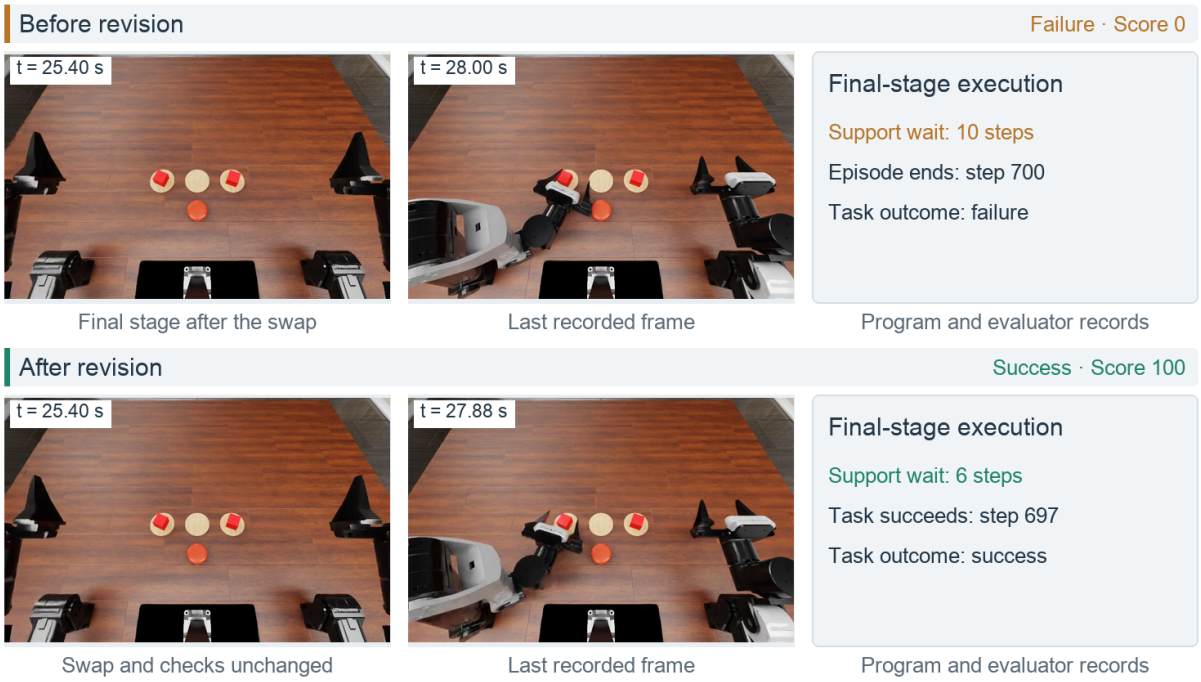


Figure 5: Execution before and after the timing revision. The top and bottom rows show the original and revised programs. The first two columns contain development-video frames; the right column reports waiting steps, termination steps, and task outcomes from the program and evaluation records. Shortening the support wait allowed the program to complete the remaining operations within its control budget.

Additional validation can also lead to rejecting a candidate. The sorting-imitation task requires the robot to observe the manipulation order of objects on the far side of the table and then move the corresponding near-side objects in that order. The agent attempted to reduce control steps by partially overlapping the two arms' handoff motions. The candidate compared this approach with parking both arms before approaching with the left arm. It accounted for 11 additional steps for observation preparation, stabilization, and redundant positioning, enabling overlapping handoff only when it was expected to save control steps overall, while retaining collision checks.

The candidate succeeded in all three initial trials. After the comparison was expanded, however, both the original and candidate programs succeeded in three of six trials, and the candidate's mean score was 55.00, below the original program's 61.67. Figure 6 shows one newly observed failure: the original program completed sorting the near-side objects, whereas the candidate stopped during demonstration tracking, before reaching the revised handoff logic. This failure therefore does not establish that the handoff revision caused the perception problem. Nevertheless, the overall comparison did not support replacing the original program. The system retained the original program and saved the candidate's test results and failure records for further diagnosis. Execution evidence thus informs both the generation of revisions and the decision to carry them into subsequent task execution.

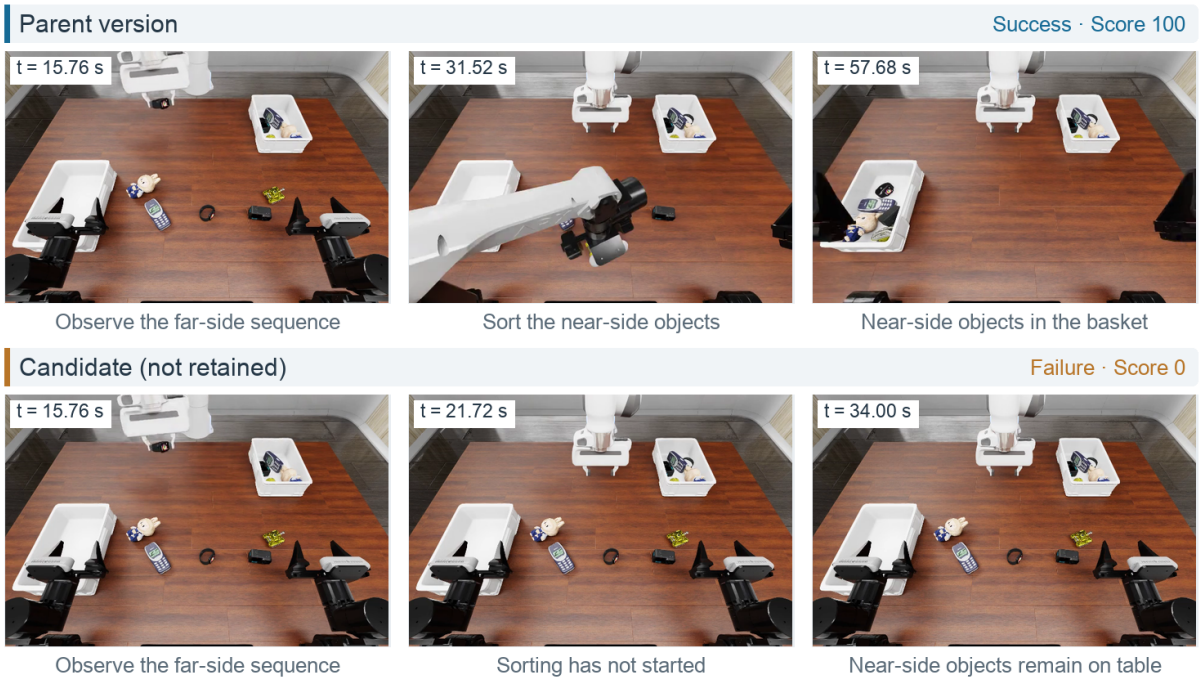


Figure 6: A candidate failure identified during additional validation. The top row shows the original program; the bottom row shows the rejected candidate. The original program continues manipulating the near-side objects and places them in the basket, whereas those objects remain on the table when the candidate stops. Execution records show that the candidate stopped during demonstration tracking, before reaching the revised handoff procedure.

6 Discussion and Limitations

The workflow depends on the available observations, perception and motion tools, and the quality of any action policy used during execution. Incomplete traces can obscure upstream causes, while contact dynamics and simulator variability can make small effects difficult to distinguish. More detailed logs help diagnosis only when their evidence can be connected to task outcomes.

The benchmark comparison evaluates the complete configuration and provides no ablations of diagnosis, hypothesis testing, version retention, or policy handoff. Overlap between development and scored test scenes further limits conclusions about generalization.

The coding agent performs much of the development, but objectives, information access, evaluation budgets, and final integration remain subject to human-defined conditions. Continued improvement also requires new, informative evidence; retaining a version or explanation does not establish its general applicability. Rejected and unresolved attempts remain part of the workflow because they help identify what should be tested next.

7 Conclusion

We describe how a coding agent develops and improves executable robot task solutions in RoboDojo. Robot observations drive program execution, while traces and native task outcomes support diagnosis, hypothesis testing, and candidate comparison between runs. Retaining both the selected program and its development evidence connects each iteration to the next.

References

[1] Jacky Liang et al. Code as Policies: Language Model Programs for Embodied Control. 2022. https://arxiv.org/abs/2209.07753.

[2] Ruiyang Si et al. Fewer Tokens, Better Action: GPT-6 Astra Robot Agents with 14% Higher Success Rate but 65% Fewer Tokens. 2026. https://arxiv.org/abs/2610.01939.

[3] RoboDojo. Benchmark and simulation leaderboard. https://robodojo-benchmark.com/leaderboard.

[4] Hongcheng Gao et al. HexaShell: A Thin Harness for Exploring the Limits of VLM Agents as Robot Coding Agents. arXiv preprint arXiv:2610.23479, 2026. https://arxiv.org/abs/2610.23479.

[5] Manifold AI Team. R^2SI: Recursive Robot Self-Improvement via a Bi-Level Agentic System with Agent–Model Co-Evolution. Technical report, 2026. https://r2si.worldscape.io/report.pdf.

[6] Wenyi Li et al. EmbodiedEvo: Self-Evolving Coding Agent For Physical Intelligence. arXiv preprint arXiv:2610.18867, 2026. https://arxiv.org/abs/2610.18867.

[7] HKU MMLab and Kinetix AI. PhysicalRSI 1.0: Recursive Self-Harness for Scaling Embodied Skills. Project page, 2026. https://mmlab.hk/research/PhysicalRSI.

[8] Wenbo Zhang et al. An Unexpected Robot Policy: Early Evaluations of GPT-6 Astra on RoboDojo and Beyond. arXiv preprint arXiv:2609.24170, 2026. https://arxiv.org/abs/2609.24170.

[9] Yanjiang Guo et al. Video Prediction Policy 2: Predict Better, Act Better. arXiv preprint arXiv:2610.10270, 2026. https://arxiv.org/abs/2610.10270.

[10] Awomo-WAM Team. Awomo-0.5: World Action Model Pre-trained at Scale. Code repository, 2026. https://github.com/Awomo-WestlakeDI/Awomo-0.5.

[11] RobotEra. VPP2-Preview. RoboDojo leaderboard entry, 2026. https://robodojo-benchmark.com/LeaderBoard.

[12] Xingyu Miao et al. InternW0-Δ: A World Action Model Bridging Predictive Dynamics and Actions with 20K+ Hours of Open Data. arXiv preprint arXiv:2609.31394, 2026. https://arxiv.org/abs/2609.31394.

[13] LiberAI. Liber-0 Lite. RoboDojo leaderboard entry, 2026. https://robodojo-benchmark.com/LeaderBoard.

[14] Haoran Wen et al. MachEmbodied-U0: Unified Understanding and Generation Model for Embodied Intelligence. arXiv preprint arXiv:2609.25627, 2026. https://arxiv.org/abs/2609.25627.

[15] Yuran Wang et al. OpenWAM: An Open, Modular Exploration Towards Systematic World-Action Model Pretraining. arXiv preprint arXiv:2609.07398, 2026. https://arxiv.org/abs/2609.07398.

[16] LiberAI. Liber-0 Preview. RoboDojo leaderboard entry, 2026. https://robodojo-benchmark.com/LeaderBoard.

[17] Wei He et al. ME-Brain-1.0: Memory, Cognition and Action for Evolving Embodied Intelligence. arXiv preprint arXiv:2609.24271, 2026. https://arxiv.org/abs/2609.24271.

[18] Simate. Simate-beta. RoboDojo policy adapter, 2026. https://github.com/XPolicyLab/XPolicyLab/tree/main/policy/Simate_beta.

[19] Futian Lab and VisIncept. Rex-M1 Preview. RoboDojo policy adapter, 2026. https://github.com/XPolicyLab/XPolicyLab/pull/131.

[20] Dexmal. DM0.5: Designed for the Open World, Where Generalization Emerges. Technical blog, 2026. https://www.dexmal.com/blog/dm0.5?lang=en-US.

[21] Yicheng Liu et al. G0.5: One Autoregressive Stream for Robot Reasoning and Action. arXiv preprint arXiv:2608.11739, 2026. https://arxiv.org/abs/2608.11739.

[22] Xiaomi Robotics Team et al. Xiaomi-Robotics-1: Scaling Vision-Language-Action Models with over 100K Hours of Real-World Trajectories. arXiv preprint arXiv:2607.15330, 2026. https://arxiv.org/abs/2607.15330.